

Omnibound.ai — Backend AI Engineer Take-Home Project

Project name: Brand Q&A Agent **Time budget:** about 8 hours of work **Deadline:** 3 days from the day you receive this

Thank you for your interest in Omnibound. This document explains, step by step, a small project we'd like you to build. It is designed to be completed in about 8 hours. If this is your first take-home project, don't worry — everything you need to know is written below. Read the whole document once before you start.

Part A — Understanding the project

A.1 What Omnibound does (in plain language)

When people research products today, they often ask an AI assistant a question such as *"What is the best project-management tool for a small team?"* instead of searching Google. The AI builds its answer from content on the web — review sites, blog posts, comparison articles, and company pages.

Omnibound is a software product used by marketing teams. It helps a company see **how their brand appears in that kind of content**: Is the brand mentioned? Is it recommended? What do writers say about it?

Every company that uses Omnibound is a separate customer. One customer must never be able to see another customer's data.

A.2 What you will build

A small backend web service, written in Python, that does two things:

1. **Ingest** — accept a set of web pages or text about some brands, split them into pieces, and store them in a vector database so they can be searched by meaning.
2. **Ask** — accept a question in plain English, find the most relevant pieces of stored text, check that they really are relevant, and produce an answer using only that text — with references back to the sources. If the stored text does not contain the answer, the service must say so rather than invent one.

This pattern is called **RAG (Retrieval-Augmented Generation)**. The “Ask” part must be built as a small **LangGraph** agent (a graph of steps with a decision point), which is explained in Part B.

A.3 Words used in this document

Word	Meaning
Brand	A company or product name, e.g. “Notion”, “HubSpot”, “Nike”.
Corpus	The set of documents you collect for the project (10–15 web pages or articles).
Tenant	One customer. Every piece of data belongs to a tenant, identified by a string <code>tenant_id</code> (e.g. <code>acme</code>).
Chunk	A piece of a document, typically 200–500 words, that gets its own entry in the vector database.
Embedding	A list of numbers that represents the meaning of a chunk. Similar meanings produce similar numbers.
Vector database	A database (e.g. Pinecone) that stores embeddings and finds the chunks whose meaning is closest to a question.
Citation	A reference back to the chunk and source URL an answer was based on.
Grading	Asking the LLM to check whether each retrieved chunk actually helps answer the question.
LangGraph	A library for building LLM workflows as a graph of nodes (steps) and edges (transitions), including conditional edges (if/else decisions).

A.4 Example of how the finished service behaves

Suppose tenant `acme` has ingested 12 articles about project-management tools.

Example 1 — normal question

Question: “What do reviewers say about Asana’s pricing compared to Trello?” The service finds chunks that mention Asana or Trello pricing, confirms they are relevant, writes a 2–3 sentence answer, and lists the chunks and URLs it used.

Example 2 — question the corpus can’t answer

Question: “What is the capital of France?” The service finds no relevant chunks, tries

once more with a rephrased question, still finds nothing, and replies that the answer is not in the corpus. It does **not** say "Paris".

Example 3 — a different tenant

Tenant `globex` (which has ingested nothing) asks the question from Example 1. The service returns "not in corpus". `globex` must never see `acme` 's data.

Part B — Technical specification

B.1 Required technologies

Area	Requirement
Language	Python 3.11 or newer
Web framework	FastAPI or Django (with Django REST Framework or Django Ninja). Choose one and say why in your video.
Agent	LangChain and LangGraph
Vector database	Pinecone (free tier is enough). Chroma, Qdrant or pgvector are acceptable if you explain the trade-off in your video.
LLM and embeddings	Any provider — OpenAI, Anthropic, Google Gemini, or a local model via Ollama. If cost is a concern, email us and we will provide an API key.

B.2 The three API endpoints

1. Ingest documents

```
POST /tenants/{tenant_id}/ingest
```

Request body:

```
{
  "documents": [
    {"url": "https://example.com/best-pm-tools-2026", "brand": "Asana"},
    {"text": "...raw article text...", "source": "manual-1", "brand": "Trello"}
  ]
}
```

Each document has **either** a `url` (you fetch it) **or** `text` (already provided), plus a `brand`

and an optional `source`.

Response body:

```
{"documents_stored": 2, "chunks_stored": 37}
```

Rule: if the same document is sent twice, it must **not** be stored twice. Compute a stable `doc_id` (for example a SHA-256 hash of the URL or the text) and use it to overwrite instead of duplicate.

2. Ask a question

```
POST /tenants/{tenant_id}/ask
```

Request body:

```
{"question": "What do reviewers say about Asana's pricing compared to Trello?"}
```

Response body:

```
{
  "answer": "Reviewers generally describe Asana as ...",
  "found_in_corpus": true,
  "citations": [
    {
      "source": "https://example.com/best-pm-tools-2026",
      "brand": "Asana",
      "chunk": 1
    }
  ],
  "trace": [
    {
      "node": "retrieve",
      "detail": "5 chunks retrieved"
    },
    {
      "node": "grade",
      "detail": "3 relevant, 2 rejected"
    },
    {
      "node": "answer",
      "detail": "answer generated from 3 chunks"
    }
  ]
}
```

When nothing relevant is found, `found_in_corpus` is `false`, `answer` explains that the corpus does not cover the question, and `citations` is empty.

3. Health check

`GET /health` — returns HTTP 200 with a small JSON showing whether the vector database and the LLM provider are reachable.

B.3 Tenant isolation (most important rule)

Data stored for one `tenant_id` must never be returned for a different `tenant_id`. In Pinecone, the simplest way is one **namespace per tenant**; alternatively use a metadata

filter on `tenant_id` in every query. You may do both. We will test this directly.

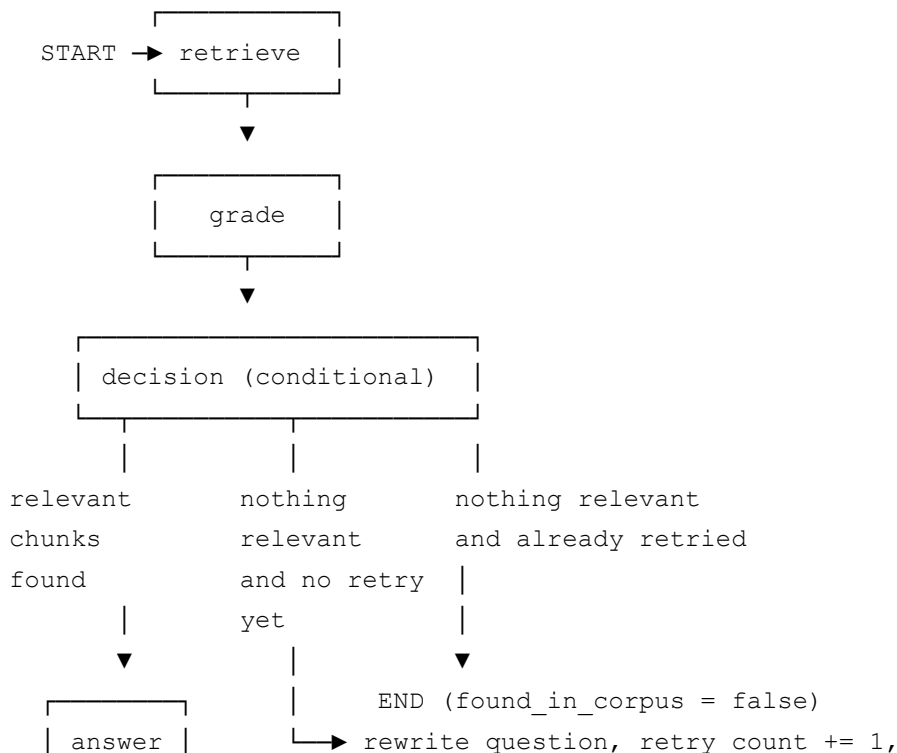
B.4 The ingestion pipeline (what happens inside `/ingest`)

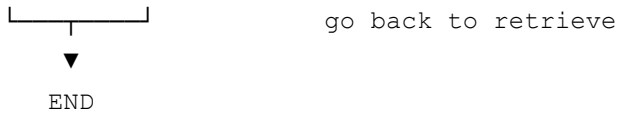
1. For each document: if it has a `url` , fetch the page and extract the readable text (BeautifulSoup, `trafilatura` , or LangChain's `WebBaseLoader` are all fine). If it has `text` , use it directly.
2. Compute `doc_id` .
3. Split the text into chunks. Choose a chunk size and overlap on purpose (for example 400 tokens with 50 overlap using `RecursiveCharacterTextSplitter`) and be ready to say in the video why.
4. Attach metadata to every chunk: `tenant_id` , `doc_id` , `chunk_index` , `source` , `brand` .
5. Create embeddings and upsert into the vector database under the tenant's namespace, using `{doc_id}-{chunk_index}` as the vector ID so re-ingesting overwrites.

B.5 The question-answering agent (what happens inside `/ask`)

Build this as a **LangGraph** `stateGraph` — not a single LangChain chain. The graph state should be a `TypedDict` or Pydantic model holding at least: `tenant_id` , `question` , `retrieved_chunks` , `relevant_chunks` , `retry_count` , `answer` , `found_in_corpus` , `trace` .

The graph has three nodes and one conditional edge:





- **retrieve** — embed the question, query the vector database for the top 5 chunks **for this tenant only**, store them in `retrieved_chunks`.
- **grade** — for each retrieved chunk, ask the LLM “Does this chunk help answer the question? yes/no”. Keep the “yes” chunks in `relevant_chunks`.
- **decision** (conditional edge) —
 - if `relevant_chunks` is not empty → go to **answer**
 - else if `retry_count == 0` → ask the LLM to rewrite the question more clearly, increase `retry_count`, go back to **retrieve**
 - else → end with `found_in_corpus = false`
- **answer** — ask the LLM to answer the question using **only** the relevant chunks, and build the `citations` list from them.

Every node must append one entry to `trace` describing what it did. This is how we (and you, when debugging) can see the path the graph took.

B.6 Code-quality expectations

- Pydantic models for every request and response body
- Use `async` for LLM calls and for fetching URLs
- All configuration (API keys, index name, model names) read from environment variables; include a `.env.example`; never commit real keys
- The service must start with one command — a `Dockerfile`, or a `Makefile` / shell script
- Two automated tests (pytest), with the LLM mocked so the tests run without an API key:
 1. **Tenant isolation** — data ingested for tenant A is not returned when tenant B asks
 2. **Decision logic** — relevant chunks → answer; no relevant chunks → retry; no relevant chunks after retry → not found
- Return clear error responses for: an unreachable URL, a tenant with no data, and an LLM timeout

Part C — Suggested step-by-step plan (about 8 hours)

You do not have to follow this order, but if you're unsure where to start, this is a sensible path.

Step 1 — Set up (30 min) Create a Git repository. Set up a Python virtual environment, install FastAPI/Django, LangChain, LangGraph, the Pinecone client, and your LLM SDK. Create a Pinecone account and one index (dimension must match your embedding model). Create `.env.example`. Make a `/health` endpoint that returns 200 and run the server once.

Step 2 — Collect a corpus (30 min, no more) Pick a product category you find interesting — project-management tools, running shoes, CRMs, coffee machines. Find 10–15 public web pages that discuss two or more brands in that category: review sites, “best of” lists, comparison articles, company pages. You may scrape them, or simply copy the text into `.txt` files under `data/` and send them as raw text. Do not polish this; it only needs to be good enough to ask questions about.

Step 3 — Build ingestion (1.5 hours) Implement `/ingest` following B.4. Test it by ingesting your corpus for tenant `acme` and checking in the Pinecone console that vectors appeared with the right metadata. Ingest the same document twice and confirm the count doesn't grow.

Step 4 — Build the graph (2.5 hours) Implement the LangGraph graph from B.5. Start with `retrieve → answer` only, get that working, then add the grade node and the conditional edge. Use `graph.get_graph().draw_ascii()` or print the trace to check the flow. Wire it to `/ask`.

Step 5 — Verify the three example behaviours (45 min) Run Example 1 (normal question), Example 2 (off-topic question), and Example 3 (different tenant) from A.4. Fix whatever doesn't match.

Step 6 — Tests and errors (1 hour) Write the two pytest tests from B.6 with the LLM mocked. Add error handling for bad URLs, empty tenant, and LLM timeouts.

Step 7 — One-command run and dry run (45 min) Add the Dockerfile or script so the service starts with one command. Then rehearse the video once, following the order in D.1, so you know every command works.

Step 8 — Record the video (45 min) Record once, following D.1. It doesn't need to be polished.

If you run out of time, stop and say so in the video. Unfinished-but-honest is fine. Rushed-and-broken is not.

Part D — What to submit

Your only submission is a video. You do not need to send us a repository, zip file or README. Keep your code — we may ask to see it in the follow-up interview.

D.1 Video walkthrough (10–12 minutes)

Record your screen with your voice using Loom, OBS, QuickTime, Zoom, or any tool. Submit it as a Loom link, an unlisted YouTube link, or a video file. Do a dry run first, then record once — it does not need to be polished.

Cover the following, in this order:

1. **Start the service** — show the one command that starts it (Docker or script) and the `/health` response.
2. **Ingest** — run `/ingest` for tenant `acme` with your corpus and show the response. Run it a second time with the same documents and show that the chunk count did not grow.
3. **Ask — normal question** — run `/ask` with a question your corpus can answer. Show the answer, the citations, and the trace.
4. **Ask — off-topic question** — run `/ask` with a question the corpus cannot answer. Show that `found_in_corpus` is `false`, the trace shows the retry, and no answer was invented.
5. **Ask — different tenant** — run the normal question as tenant `globex` and show that it returns nothing.
6. **Walk through the code** — open your project and briefly show:
 - the project structure and which web framework you chose, and why
 - the ingestion code: how you chunk (size/overlap and why) and what metadata each chunk carries
 - the LangGraph graph: the state, the three nodes, and the conditional edge
 - how tenant isolation is implemented
7. **Run the tests** — run `pytest` and show the two tests passing without an API key.
8. **Reflect** — explain one trade-off you made, one thing that is rough or unfinished, and one thing you would do with more time. Honesty here is scored positively.

Keep the code visible on screen while you talk about it. Please do not show an API key or `.env` file in the recording.

D.2 Submission checklist

- Video is 10–12 minutes, screen plus voice
 - Shows the service starting with one command and `/health`
 - Shows `/ingest` twice with no duplicate chunks
 - Shows `/ask` with a normal question, an off-topic question, and a second tenant
 - Shows the ingestion code, the graph code, and tenant isolation
 - Shows the two pytest tests passing
 - Ends with the trade-off / rough edges / next-steps reflection
 - No API keys or secrets visible in the recording
-

Part E — How we evaluate

Area	Weight	What we look for
RAG quality	25%	Sensible chunking and metadata; retrieval surfaces the right passages; honest “not found” behaviour
Agent design	25%	Real LangGraph state and conditional edge; answers use only retrieved chunks; readable trace
API and code quality	20%	Clean structure, Pydantic models, async used properly, the two tests, error handling
Multi-tenancy and production sense	15%	Isolation works, idempotent ingest, env-based config, one-command run
Communication	15%	Clear, well-organised video walkthrough; honesty about limitations

Part F — Ground rules

- You may use any AI coding assistant. In the follow-up interview we will ask you to explain your code line by line, so make sure you understand everything you submit.
- Stop at around 8 hours. Time spent beyond that does not earn extra credit.
- If anything in this document is unclear, email us. Asking a good question is a positive

signal, not a negative one.